

Arquitetura do Compilador mcc (Mancha C Compiler)

Análise da Pasta src do Compilador e Comparativo com o Simulador do Processador Mancha

Este documento detalha o funcionamento interno dos arquivos da pasta `src` do **Compilador mcc** (Mancha C Compiler). O `mcc` é responsável por traduzir programas escritos em linguagem **C (subconjunto da linguagem)** diretamente para código de montagem (**Assembly Mancha .asm**). Além disso, apresenta-se uma comparação fundamental entre o papel do **Compilador** e o do **Simulador** na cadeia de ferramentas.

Diferença Fundamental: Simulador vs. Compilador

Na arquitetura do sistema Mancha, o **Compilador** e o **Simulador** atuam em pontas totalmente distintas do processo de desenvolvimento e execução de software:

Aspecto	Compilador (mcc) — Arquivos Atuais	Simulador (simulador) — Arquivos Anteriores
Papel Principal	Tradutor de Alto Nível: Converte código-fonte C legível para humanos em Assembly Mancha (.asm).	Executor de Hardware: Emula o ciclo de instruções, a CPU de 16 bits, registradores e memória física RAM em tempo de execução.
Entrada (Input)	Arquivos de código-fonte em linguagem C (.c e .h).	Arquivo binário/objeto pré-montado (.mob / mob.j1) e imagens de disco.
Saída (Output)	Arquivo em Linguagem de Montagem Assembly Mancha (.asm).	Execução em tempo real, estado visual da CPU (ncurses) e saídas de terminal/disco.
Tempo de Atuação	Tempo de Compilação (Compile Time): Roda na máquina hospedeira (PC/Linux) antes do programa ser executado.	Tempo de Execução (Runtime): Roda enquanto o programa simulado está ativamente processando instruções.
Estruturas Chave	Tabela de símbolos C, AST de expressões, gerador de pré-processador, analisador léxico/sintático.	ALU, MMU (Segmentação/Paginação), pilha de supervisor, registradores (r0-r7, s0-s7), portas de E/S.

Como o Compilador `mcc` Funciona (Pipeline de Compilação)

Fluxo de Transformação de Código no `mcc`:

1. Pré-Processamento
- pre.c resolve #include, #define, #ifdef e remove comentários.
2. Análise Léxica
- lexer.c converte a string de C expandida em um vetor de Tokens estruturados.
3. Análise Tipos/Símbolos
- tipos.c e simbolos.c mantêm tabelas de variáveis locais, globais, structs e ponteiros.
4. Parser & Emissão
- parser.c analisa gramática (recursivo descendente) em 2 passadas e gera o código Assembly Mancha (.asm).
5. Ponto de Entrada
- main.c orquestra todas as fases e salva o arquivo .asm final.

[fonte: 21] `pre.c`

Pré-processador C

Processador de Diretivas (`#include`, `#define`, `#if`, `#ifdef`, macros)

O `pre.c` implementa um pré-processador C completo. Ele varre o arquivo fonte C resolvendo inclusões de cabeçalhos (`#include`), expansão de macros simples e funcionais com parâmetros (`#define FOO(x)`), controle condicional (`#if`, `#ifdef`, `#elif`, `#else`, `#endif`) e substituição de comentários por espaços. Ele gera marcadores especiais `#LINHA` num "arquivo" para manter a rastreabilidade de erros durante a compilação.

PRINCIPAIS RESPONSABILIDADES E FUNÇÕES:

- `pre_processa_arquivo()` : Função principal do pré-processador que inicia a leitura recursiva e devolve o texto expandido.
- `expande_texto()` : Realiza a substituição textual de macros simples e funcionais, evitando expansão infinita recursiva.
- `processa_diretiva()` : Identifica e executa diretivas iniciadas por `#`.
- `avalua_if()` : Avaliador de expressões booleanas e relacionais para comandos `#if` e `#elif`.

[fonte: 18] `lexer.c`

Analisador Léxico

Tokenizador (Transforma código texto em Tokens isolados)

O `lexer.c` é responsável por transformar o fluxo contínuo de caracteres produzido pelo pré-processador em uma sequência estruturada de **tokens** (palavras-chave como `int`, `if`, `while`; identificadores; numerais literais; strings e operadores relacionais/aritméticos como `==`, `->`, `+=`). Ele também decodifica sequências de escape em literais (como `\n`, `\t`, `\xHH`).

PRINCIPAIS RESPONSABILIDADES E FUNÇÕES:

- `lexer_tokeniza()` : Percorre o buffer de texto e preenche o vetor de tokens (`token_t`) identificando tipos como `TK_IDENT`, `TK_INT_LIT`, `TK_STRING_LIT`, etc.
- `consume_espacos_e_marcadores()` : Ignora espaços em branco e atualiza a linha/arquivo atual ao ler os marcadores `#LINHA` emitidos pelo `pre.c`.
- `decodifica_escape()` : Converte caracteres mascarados dentro de strings/chars para seus bytes reais em ASCII.

Analizador Sintático Recursivo Descendente e Emissor de Assembly Mancha

O `parser.c` é o coração do compilador. Ele consome a lista de tokens e valida a sintaxe da linguagem C. Para expressões, constrói pequenas árvores sintáticas para resolver prioridades de operadores e lvalues/rvalues. O compilador utiliza uma técnica de **duas passadas por função**: a 1ª passada ("seca") varre as declarações para calcular o tamanho exato do quadro de pilha local (stack frame); a 2ª passada ("real") emite o código em Assembly Mancha (`ld`, `st`, `add`, `sub`, `jmpc`, `call`, `ret`).

PRINCIPAIS RESPONSABILIDADES E MECANISMOS:

- `compilador_compila()` : Ponto de entrada que coordena a compilação das declarações globais, tabela de strings e funções.
- `compila_corpo_funcao()` : Executa as duas passadas por função (cálculo de deslocamento em `BP` e geração real de instruções).
- `gera_valor()` / `gera_endereco()` : Geram o código assembly para carregar valores no registrador `r0` ou calcular endereços para operações com ponteiros, vetores e campos de structs.
- `carrega()` / `guarda()` : Emitem instruções `ld` / `ldb` e `st` / `stb` tratando acesso a variáveis locais no frame da pilha (`bp+N`).

Gerenciador do Sistema de Tipos da Linguagem C

O `tipos.c` mantém e valida os tipos de dados do programa em C (`int`, `char`, `void`, ponteiros `T*`, vetores `T[N]` e estruturas `struct`). Ele calcula o tamanho em bytes ocupado por cada tipo (ex: `int` = 2 bytes no Mancha, `char` = 1 byte, ponteiro = 2 bytes) e gerencia o layout de memória de structs (offsets de membros).

PRINCIPAIS RESPONSABILIDADES E FUNÇÕES:

- `tipo_tamanho()` : Devolve o tamanho exato em bytes de um tipo para cálculo de ponteiros e alocação.
- `estrutura_adiciona_membro()` : Calcula o offset relativo acumulado de cada campo em uma `struct`.
- `tipo_decai()` : Implementa a regra do C onde nomes de arrays em expressões decaem automaticamente para ponteiros do tipo base.

Gerenciador de Escopo de Variáveis e Funções

O `simbolos.c` implementa a Tabela de Símbolos e o controle de escopo (globais vs. locais). Ele permite declarar e consultar variáveis e funções, garantindo que variáveis locais em blocos internos ocultem variáveis globais de mesmo nome e que parâmetros de função sejam mapeados para posições corretas da pilha.

PRINCIPAIS RESPONSABILIDADES E FUNÇÕES:

- `simbolo_acha()` : Busca um símbolo primeiro no escopo local corrente (pilha de escopos) e, se não encontrar, busca na tabela global.
- `global_declara_var()` / `global_declara_funcao()` : Cria identificadores globais com rótulos de montador (ex: `_g_var` ou `_f_func`).
- `escopo_marca()` / `escopo_volta()` : Gerencia o ciclo de vida de escopos aninhados delimitados por chaves `{ ... }`.

Controlador da Linha de Comando e Pipeline de Compilação

O `main.c` é o ponto de entrada da ferramenta `mcc`. Ele processa os argumentos de linha de comando (como `-o` saída.asm e `-I` diretorio), aciona sequencialmente o pré-processador, o lexer e o parser, e grava o arquivo final em assembly Mancha.

PRINCIPAIS RESPONSABILIDADES:

- Processamento de flags de linha de comando (`-o` , `-I`).
- Geração de um prefixo único baseado em timestamp/PID para evitar colisão de rótulos internos durante a montagem final.
- Abertura e escrita do arquivo final de saída `.asm` .